

SOFTWARE REQUIREMENTS SPECIFICATION

Charity Management System

Single-Charity Web Application

Document Field	Value
Document type	Software Requirements Specification (SRS)
Version	1.0
Date	16 July 2026
Status	Production-readiness baseline
Technology baseline	Java 17, Spring Boot, Spring Security, Thymeleaf, PostgreSQL, Docker
Deployment baseline	Render web service with Neon PostgreSQL cloud database

Purpose

This document defines the functional, non-functional, data, security, interface, and deployment requirements for the Charity Management System. It serves as a shared reference for development, testing, deployment, maintenance, and future enhancement.

Table of Contents

1. Introduction
2. Overall Description
3. User Roles and Permissions
4. Functional Requirements
5. External Interface Requirements
6. Data Requirements
7. Non-Functional Requirements
8. Business Rules
9. Use Cases
10. Acceptance Criteria
11. ER Diagram
12. Assumptions and Future Scope

1. Introduction

1.1 Purpose

The Charity Management System centralizes the operational and financial records of one charity organization. The system provides authenticated users with role-appropriate access to members, annual records, budgets, donations, projects, events, and reports.

1.2 Scope

The application is a server-rendered web system intended for authorized charity personnel. It supports daily administration, financial traceability, participation management, historical review, and exportable reporting. Public self-registration may be displayed for demonstration purposes but can be disabled in the hosted production environment.

1.3 Objectives

- Maintain one authoritative source of truth for charity operations.
- Protect sensitive operations through authentication and role-based authorization.
- Track donations, budgets, projects, events, members, and yearly activity.
- Provide searchable, paginated, and exportable records.
- Support reliable cloud deployment and administrator-led maintenance.

1.4 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification.
RBAC	Role-Based Access Control.
CRUD	Create, Read, Update, and Delete operations.
HEAD	Highest operational role with full administrative authority.
SUBHEAD	Deputy administrator with broad management permissions.
TREASURER	Financial role responsible for budgets and donations.
MEMBER	Standard authenticated user with limited access.
Year record	A yearly container that groups financial and operational data.
PDF report	A generated, downloadable document representing selected system data.

2. Overall Description

2.1 Product Perspective

The system follows a layered Spring Boot architecture consisting of web controllers, DTOs, service interfaces and implementations, repositories, JPA entities, security configuration, Thymeleaf templates, and PostgreSQL persistence. It is containerized with Docker and designed for cloud deployment.

2.2 Product Functions

- User authentication and account administration.
- Role-based authorization and access-denied handling.

- Member management.
- Year and annual record management.
- Budget planning and monitoring.
- Donation registration and financial traceability.
- Project planning and member assignment.
- Event planning and member assignment.
- Search, filtering, sorting, and pagination.
- PDF generation and export.
- Application logging, validation, error handling, and cloud operation.

2.3 Operating Environment

Layer	Environment
Client	Modern desktop or mobile browser with JavaScript and cookies enabled.
Application	Java 17 and Spring Boot web application.
Presentation	Thymeleaf, HTML5, CSS3, and JavaScript.
Security	Spring Security with BCrypt password hashing.
Persistence	PostgreSQL using Spring Data JPA/Hibernate.
Deployment	Docker container hosted on Render.
Cloud database	Neon PostgreSQL accessed through environment variables.
Source control	Git repository, typically hosted on GitHub.

2.4 Constraints

- The current release supports one charity organization.
- All protected pages require authentication.
- Access permissions are determined by the assigned role.
- The production database must not rely on mock-data initialization.
- Secrets and connection strings must be supplied through environment variables.
- Deleting records with dependencies must be prevented, cascaded safely, or confirmed according to business rules.

3. User Roles and Permissions

Capability	HEAD	SUBHEAD	TREASURER	MEMBER
View dashboard and records	Yes	Yes	Yes	Yes
Manage user accounts and roles	Yes	Limited/No	No	No
Create/update/delete members	Yes	Yes	Limited	No
Manage years	Yes	Yes	View	View
Manage budgets	Yes	Yes	Yes	View
Manage donations	Yes	Yes	Yes	View
Manage projects	Yes	Yes	Limited	View
Manage events	Yes	Yes	Limited	View
Assign members to projects/events	Yes	Yes	Limited	No
Export reports	Yes	Yes	Yes	As permitted

Note: The final permission matrix is enforced by method-level and URL-level Spring Security rules.
“Limited” means the organization may configure narrower permissions without changing the core SRS.

4. Functional Requirements

4.1 Authentication and User Accounts

FR-AUTH-01 - Login: The system shall authenticate users using a username or email and password.
[Priority: High]

FR-AUTH-02 - Password protection: The system shall store passwords only as BCrypt hashes and shall never expose plaintext passwords. [Priority: High]

FR-AUTH-03 - Logout: The system shall allow an authenticated user to terminate the session securely.
[Priority: High]

FR-AUTH-04 - Disabled registration: The system shall allow public registration to be disabled in production while retaining the registration page for demonstration purposes. [Priority: High]

FR-AUTH-05 - Role assignment: Only an authorized administrator shall assign or change user roles.
[Priority: High]

FR-AUTH-06 - Access denied: Unauthorized operations shall redirect to or display a styled access-denied page without performing the action. [Priority: High]

4.2 Member Management

FR-MEM-01 - List members: Display members with pagination and search/filter controls. [Priority: High]

FR-MEM-02 - Create member: Allow authorized users to create a member after validation. [Priority: High]

FR-MEM-03 - View member: Show member details and related participation or donations where available. [Priority: High]

FR-MEM-04 - Update member: Allow authorized users to edit member information. [Priority: High]

FR-MEM-05 - Delete member: Allow safe deletion only when referential integrity rules are satisfied.
[Priority: High]

FR-MEM-06 - Activation status: Support active/inactive member status. [Priority: High]

4.3 Year Management

FR-YEAR-01 - List years: Display annual records in a paginated list. [Priority: High]

FR-YEAR-02 - Create year: Create a unique year record. [Priority: High]

FR-YEAR-03 - View year summary: Display the year budget, donations, projects, events, and members involved. [Priority: High]

FR-YEAR-04 - Update year: Allow authorized correction of year metadata. [Priority: High]

FR-YEAR-05 - Delete year: Prevent unsafe deletion when dependent records exist or handle them according to configured cascade rules. [Priority: High]

FR-YEAR-06 - Export year report: Generate a pdf summary for a selected year. [Priority: High]

4.4 Budget Management

FR-BUD-01 - Create budget: Create one budget for a selected year. [Priority: High]

FR-BUD-02 - Track amounts: Store allocated, spent, and remaining amounts. [Priority: High]

FR-BUD-03 - Validate values: Reject negative amounts and inconsistent totals. [Priority: High]

FR-BUD-04 - Update budget: Allow authorized financial users to update the budget. [Priority: High]

FR-BUD-05 - View budget: Show budget data in yearly and budget-specific views. [Priority: High]

4.5 Donation Management

FR-DON-01 - Record donation: Store donor, amount, date, payment method, notes, year, and optional project/member links. [Priority: High]

FR-DON-02 - Validate donation: Require a positive amount and valid date. [Priority: High]

FR-DON-03 - Search donations: Filter donations by donor, date, year, project, or other supported criteria. [Priority: High]

FR-DON-04 - Update donation: Allow authorized users to correct a donation. [Priority: High]

FR-DON-05 - Delete donation: Allow authorized deletion with confirmation and logging. [Priority: High]

FR-DON-06 - Aggregate donations: Calculate totals by year, project, member/donor, and date range where supported. [Priority: High]

4.6 Project Management

FR-PROJ-01 - Create project: Store project details, dates, status, planned budget, and year. [Priority: High]

FR-PROJ-02 - Assign members: Support many-to-many member assignment to projects. [Priority: High]

FR-PROJ-03 - Update project: Allow authorized changes to project details and assignments. [Priority: High]

FR-PROJ-04 - Delete project: Remove a project safely and clean join-table records. [Priority: High]

FR-PROJ-05 - Project summary: Display assigned members, donations, dates, budget, and status. [Priority: High]

4.7 Event Management

FR-EVT-01 - Create event: Store event details, date, location, status, and year. [Priority: High]

FR-EVT-02 - Assign members: Support many-to-many member assignment to events. [Priority: High]

FR-EVT-03 - Update event: Allow authorized changes to event details and assignments. [Priority: High]

FR-EVT-04 - Delete event: Remove an event safely and clean join-table records. [Priority: High]

FR-EVT-05 - Event summary: Display participants and event information. [Priority: High]

4.8 Search, Pagination, and Reporting

FR-SYS-01 - Pagination: Paginate large entity lists and preserve active filters during navigation. [Priority: High]

FR-SYS-02 - Search and filtering: Provide case-insensitive and entity-appropriate filtering. [Priority: High]

FR-SYS-03 - Validation feedback: Display clear field-level and global validation messages. [Priority: High]

FR-SYS-04 - PDF export: Produce readable pdf reports with correct totals and selected data. [Priority: High]

FR-SYS-05 - Empty states: Show meaningful empty-state messages when no data exists. [Priority: High]

FR-SYS-06 - Audit logging: Log authentication failures and sensitive create/update/delete operations without logging passwords or secrets. [Priority: High]

5. External Interface Requirements

5.1 User Interface

- Responsive navigation bar and consistent footer.
- Dashboard or home page with links to major modules.
- Consistent cards, forms, tables, action buttons, validation messages, and confirmation flows.
- Access-denied and error pages styled consistently with the application.
- Keyboard-accessible controls, visible focus states, descriptive labels, and sufficient contrast.

5.2 Software Interfaces

Interface	Requirement
PostgreSQL	The application shall use a JDBC connection configured by environment variables.
Spring Security	Authentication and authorization shall be enforced centrally and at method level where required.
PDF library	The report service shall generate standards-compliant downloadable PDF files.
Render	The application shall expose the configured HTTP port and support automated deployment from the source repository or Docker image.
Neon	The database connection shall support SSL as required by the provider.

5.3 Communications

All production traffic shall use HTTPS. Session cookies shall use secure attributes where supported. Database traffic shall use encrypted provider-supported connections.

6. Data Requirements

6.1 Core Entities

Entity	Purpose	Key relationships
UserAccount	Authentication identity, role, and status.	Optionally linked one-to-one with Member.
Member	Charity participant or volunteer profile.	Projects, events, and optional donations.
YearRecord	Annual grouping and reporting context.	Budget, projects, events, and donations.
Budget	Annual financial plan and totals.	Exactly one YearRecord.
Project	Charity initiative.	Year, members, and optional donations.

Event	Scheduled charity activity.	Year and members.
Donation	Incoming financial contribution.	Year and optional project/member.
ProjectMember	Join entity/table for project participation.	Project and Member.
EventMember	Join entity/table for event participation.	Event and Member.

6.2 Integrity and Retention

- Primary keys shall uniquely identify all records.
- Usernames, user emails, and year values shall be unique where configured.
- Foreign keys shall preserve valid relationships.
- Monetary values shall use fixed-precision decimal types rather than floating-point types.
- Timestamps shall be stored consistently and displayed in the configured local timezone.
- Production backups and retention shall follow the cloud database plan and charity policy.

7. Non-Functional Requirements

ID	Category	Requirement
NFR-SEC-01	Security	Protected data and actions shall require authentication and authorization.
NFR-SEC-02	Security	Passwords, secrets, database URLs, and credentials shall not be committed to source control.
NFR-SEC-03	Security	The application shall validate input and rely on parameterized persistence operations.
NFR-PERF-01	Performance	Normal list and detail pages should respond within 3 seconds under expected single-charity load, excluding cold starts and external outages.
NFR-PERF-02	Performance	Large result sets shall use server-side pagination.
NFR-REL-01	Reliability	Committed transactions shall preserve data consistency; failed multi-step updates shall roll back.
NFR-REL-02	Reliability	The application shall recover after container restart without data loss in the external database.
NFR-USA-01	Usability	The interface shall be consistent, readable, responsive, and provide clear success/error feedback.
NFR-MNT-01	Maintainability	The code shall retain a layered architecture and separate DTO, service, repository, security, and web concerns.
NFR-MNT-02	Maintainability	Configuration shall be environment-specific and externalized.
NFR-COMP-01	Compatibility	The system shall support current versions of Chrome, Edge, Firefox, and Safari.
NFR-OBS-01	Observability	Operational logs shall support diagnosis without exposing sensitive

		values.
NFR-ACC-01	Accessibility	Forms and navigation should conform to WCAG 2.1 AA principles where practical.

8. Business Rules

BR-01 - Rule: Only authenticated users may access protected modules. [Priority: High]

BR-02 - Rule: Only the HEAD role may perform unrestricted account and role administration. [Priority: High]

BR-03 - Rule: Each year value must be unique. [Priority: High]

BR-04 - Rule: A year may have at most one budget. [Priority: High]

BR-05 - Rule: Donation amounts and budget amounts must be non-negative; donation amounts must be greater than zero. [Priority: High]

BR-06 - Rule: Project end dates must not precede project start dates. [Priority: High]

BR-07 - Rule: Member assignments must not create duplicate project-member or event-member pairs. [Priority: High]

BR-08 - Rule: An unauthorized create, update, delete, or assignment request must be rejected even when submitted directly to the endpoint. [Priority: High]

BR-09 - Rule: Deleting an entity must not leave orphaned join-table or dependent records. [Priority: High]

BR-10 - Rule: Generated reports must use persisted data and calculated totals current at generation time. [Priority: High]

9. Representative Use Cases

Use case	Primary actor	Precondition	Main result
UC-01 Log in	Any user	Active account exists.	Authenticated session is created and role-appropriate navigation is shown.
UC-02 Register donation	HEAD/SUBHEAD/ TREASURER	User is authenticated and a year exists.	Validated donation is stored and totals are updated in views/reports.
UC-03 Create project and assign members	HEAD/SUBHEAD	Project year and members exist.	Project is created and selected members are linked without duplicates.
UC-04 Create event and assign members	HEAD/SUBHEAD	Event year and members exist.	Event is created and selected members are linked.
UC-05 Review annual activity	Authorized user	Year record exists.	User sees budget, donations, projects, events, and participation.
UC-06 Export annual report	Authorized user	Year data exists.	A readable PDF is generated and downloaded.
UC-07 Attempt forbidden action	MEMBER or insufficient role	User is authenticated.	Action is rejected and access-denied feedback is displayed; no data changes

			occur.
--	--	--	--------

10. Acceptance Criteria

- All functional requirements marked High are implemented and tested.
- Role-based permissions are verified using direct URL/HTTP requests as well as UI controls.
- CRUD operations preserve database integrity.
- Filters and pagination work together and retain query parameters.
- PDF reports open successfully and contain accurate totals.
- The Docker image builds successfully and the application starts with environment-provided configuration.
- The Render deployment connects to Neon using production credentials and HTTPS.
- No mock production data is inserted automatically.
- A documented administrator account creation or initialization procedure exists.
- Critical paths pass manual or automated regression testing before release.

11. Entity-Relationship Diagram

The diagram below is a logical data model derived from the implemented modules and known relationships. Nullable links allow anonymous/external donors and donations not restricted to a project. Exact physical column names may be aligned with the final JPA entities.

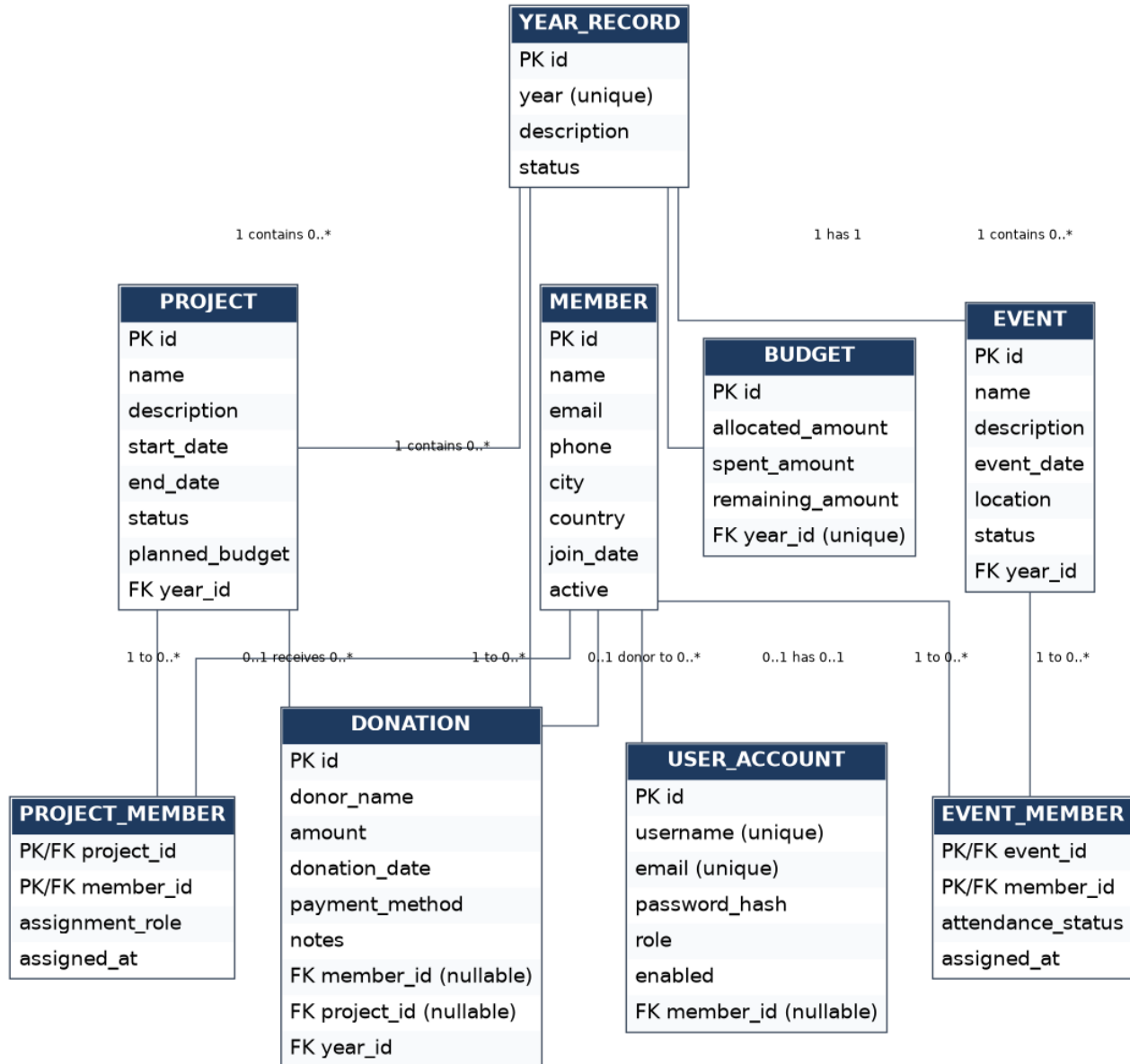


Figure 1. Logical ER diagram for the Charity Management System.

12. Assumptions and Future Scope

12.1 Assumptions

- The application currently serves one charity organization.
- User accounts are created or enabled under administrator control in production.
- The charity defines its own retention, backup, and financial approval policies.
- Currency display is configured consistently for the charity.
- Exact permission details may be tightened while preserving the role hierarchy described here.

12.2 Future Scope

- Multi-charity tenancy with isolated users, members, projects, events, budgets, and reports.
- Google OAuth 2.0 / OpenID Connect login.
- Email notifications and invitation workflows.
- Spreadsheet export and richer analytics dashboards.
- Audit-history screens and approval workflows for financial changes.
- Cloud object storage for receipts and supporting documents.
- Public donation portal and payment-provider integration.
- API integration with external accounting, CRM, or reporting systems.

Document Approval

Role	Name	Signature	Date
Project owner			
Developer			
Charity representative			